

Week 5 - Data-Driven Testing (Excel & CSV)

Duration: 4 Hours (2h teaching + 2h practice) **Focus:** One test, many inputs — driving the same flow with N rows of test data so you cover boundary cases, equivalence classes, and real-world scenarios without copy-paste.

Prerequisites:

- Week 1-4 completed (you have your own framework with BasePage, page objects, Mocha + Chai, mochawesome reports, and dotenv config)
- Comfortable creating new *.spec.js files in tests/
- Basic spreadsheet skills (you can open Excel or Google Sheets and edit cells)

Setup Instructions

1. Move into your Week 4 framework

You should already have a working framework from Week 4. We're going to extend it — not start over.

```
cd path/to/your-week4-framework
```

If you have an open mochawesome report from last week, that's fine — we'll be regenerating it.

2. Install the data libraries

We'll use two new packages this week:

```
npm install xlsx csv-parse
```

- **xlsx** — reads and writes Excel files (.xlsx, .xls, and even .csv). One of the most-used spreadsheet libraries in Node.
- **csv-parse** — focused, robust CSV parser. Handles quoted fields, embedded commas, BOM characters, custom delimiters.

Optional alternative (we won't use it today, but worth knowing):

```
# npm install exceljs
```

exceljs is more powerful for *writing* Excel files (formatting, formulas, multiple sheets). For *reading*, xlsx is simpler and faster. Pick one and stick with it.

3. Verify the install

```
npm ls xlsx csv-parse
```

You should see both packages listed without (empty) warnings.

4. Create the data folder

```
mkdir data
```

This is where Excel and CSV test fixtures live. Add a note to your README that anything sensitive (real user credentials, prod data) should be .gitignore-d.

1. Why Data-Driven Testing Matters

The Problem: Tests That Repeat Themselves

Imagine you've written a login test for your Week 4 framework. It looks like this:

```
// tests/login.spec.js
it("should log in with valid credentials", async () => {
  await loginPage.login("standard_user", "secret_sauce");
  expect(await driver.getCurrentUrl()).to.include("/inventory");
});
```

Now QA tells you to also test:

- An invalid password
- An empty username
- A locked-out user

- A user with special characters in the password
- A user with leading/trailing spaces (browser autofill quirk)
- 5 more edge cases the security team thought of last sprint

The naive solution is copy-paste:

```
it("should fail with invalid password", async () => {
  /* ... */
});
it("should fail with empty username", async () => {
  /* ... */
});
it("should fail when user is locked out", async () => {
  /* ... */
});
// ... 7 more times ...
```

You now have ~10 nearly identical tests. When the Login API changes (say, the error message wording), you fix it in 10 places. When QA wants to add 20 more cases, you copy-paste 20 more times.

The Solution: One Test, Many Inputs

Data-driven testing inverts the relationship. The **flow** is written once. The **inputs and expected outcomes** live in a spreadsheet (or CSV). Mocha generates one `it` block per row at runtime.

```
const rows = readExcel("data/login-data.xlsx");

rows.forEach((row) => {
  it(`[${row.testId}] ${row.testCase}`, async () => {
    await loginPage.login(row.username, row.password);
    if (row.expected === "pass") {
      expect(await driver.getCurrentUrl()).to.include("/inventory");
    } else {
      expect(await loginPage.getErrorMessage()).to.include(row.expectedError);
    }
  });
});
```

Add a row to the spreadsheet → you get a new test. Delete a row → you lose a test. The test code doesn't change.

Real-World Wins

Concern	Hardcoded approach	Data-driven approach
Adding a new edge case	Copy-paste a <code>it</code> block	Add a row to the file
Non-developer adds tests	Has to learn JS + Mocha	Edits a spreadsheet
Localization testing	One test per locale	One file per locale, same code
Boundary coverage (0, 1, max, max+1)	4 tests, all duplicated	4 rows
Security teams scanning for inputs	Hunt through code	Read one CSV

Equivalence Classes & Boundary Values (a quick refresher)

Two test-design ideas your Level 1 course probably mentioned:

- **Equivalence class:** A set of inputs that should all behave the same. Example: "any number between 1 and 100 is valid for quantity." You don't test all 100 — you test one.
- **Boundary values:** Inputs at the edges. For "1 to 100," test 0, 1, 100, 101. Bugs cluster at boundaries.

Data-driven tests are *the* natural home for these. Each equivalence class gets one row; each boundary gets a row. The spreadsheet becomes a checklist of what's covered.

2. Reading Excel with the `xlsx` Package

The 4-Line Pattern

Once you know this, every Excel reading task in Node looks the same:

```
const XLSX = require("xlsx");

const workbook = XLSX.readFile("data/login-data.xlsx");
const sheet = workbook.Sheets[workbook.SheetNames[0]];
const rows = XLSX.utils.sheet_to_json(sheet);
```

That's it. `rows` is now an array of plain JavaScript objects. The keys are the column headers (from row 1 of the sheet); the values are the cell contents.

What the Spreadsheet Looks Like

Here's a sample `login-data.xlsx` (rendered as a markdown table for the notes — in real life it's a `.xlsx` file):

testId	testCase	username	password	expected	expectedError
L01	valid standard user	standard_user	secret_sauce	pass	
L02	valid problem user	problem_user	secret_sauce	pass	
L03	locked out user	locked_out_user	secret_sauce	fail	locked out
L04	wrong password	standard_user	wrong_pass	fail	do not match
L05	empty username		secret_sauce	fail	Username is required
L06	empty password	standard_user		fail	Password is required
L07	username with spaces	standard_user	secret_sauce	fail	do not match
L08	sql injection attempt	' OR '1'='1	x	fail	do not match
L09	unicode username	用户	secret_sauce	fail	do not match
L10	very long password	standard_user	aaaaaaaaaaaaaaaa...	fail	do not match
L11	xss attempt	<script>x</script>	secret_sauce	fail	do not match
L12	trailing newline in pass	standard_user	secret_sauce\n	fail	do not match

After `sheet_to_json(sheet)`, the variable `rows` is:

```
[
  {
    testId: "L01",
    testCase: "valid standard user",
    username: "standard_user",
    password: "secret_sauce",
    expected: "pass",
    expectedError: undefined,
  },
  { testId: "L02", testCase: "valid problem user", username: "problem_user" /* ... */ },
  // ... 10 more rows ...
];
```

Reading a Specific Sheet

By default we used the first sheet. To read a named sheet:

```
const sheet = workbook.Sheets["LoginCases"]; // sheet tab name
const rows = XLSX.utils.sheet_to_json(sheet);

workbook.SheetNames is an array of all tab names — useful for iterating multiple sheets:

for (const name of workbook.SheetNames) {
  const sheet = workbook.Sheets[name];
  const rows = XLSX.utils.sheet_to_json(sheet);
  console.log(`Sheet ${name}: ${rows.length} rows`);
}
```

Wrap It in a Helper

Following our framework conventions, put readers in `utils/`:

```
// utils/excel.js
const XLSX = require("xlsx");

function readExcel(filepath, sheetName) {
  const workbook = XLSX.readFile(filepath);
  const sheet = sheetName ? workbook.Sheets[sheetName] : workbook.Sheets[workbook.SheetNames[0]];
  if (!sheet) {
    throw new Error(`Sheet not found: ${sheetName || "(first sheet)"} in ${filepath}`);
  }
  return XLSX.utils.sheet_to_json(sheet, { defval: "" });
}

module.exports = { readExcel };
```

The `{ defval: "" }` option fills empty cells with `""` instead of leaving the key off entirely. Without it, an empty username cell means `row.username` is `undefined`, which can mask bugs in your assertions.

Gotchas with Excel Auto-Typing

xlsx infers types from cell values. This bites you:

- A cell that looks like 00123 (a leading-zero zip code) becomes the **number** 123. Excel does the damage; the lib just reports what's stored.
- A cell with =A1+1 returns the **computed value**, not the formula.
- Empty cells become undefined keys (use default to make them "").
- Date cells return JS Date objects (or weird Excel serial numbers depending on version) — be careful.

Fix for leading zeros: in Excel, format the column as **Text** before typing, or prefix with a single quote ('00123) so Excel stores it as a string.

3. Reading CSV Files

Approach 1: Built-in fs (only for trivial cases)

If your CSV is dead simple — no quoted fields, no commas inside values, no special chars — you can split it yourself:

```
const fs = require("fs");

const text = fs.readFileSync("data/users.csv", "utf8");
const lines = text.trim().split("\n");
const headers = lines[0].split(",");
const rows = lines.slice(1).map((line) => {
  const values = line.split(",");
  const row = {};
  headers.forEach((h, i) => {
    row[h] = values[i];
  });
  return row;
});
```

Don't use this in real projects. The moment a row contains "Smith, John" (a name with a comma in quotes) or a Windows line ending or a BOM character, this breaks silently. Reach for a real parser.

Approach 2: csv-parse (recommended)

```
const fs = require("fs");
const { parse } = require("csv-parse/sync");

const text = fs.readFileSync("data/users.csv", "utf8");
const rows = parse(text, {
  columns: true, // first row is headers; output objects keyed by header
  skip_empty_lines: true,
  trim: true, // trim whitespace around values
});
```

rows is now [{ testId: "...", username: "...", ... }, ...] — same shape as the xlsx version.

Note the csv-parse/sync import path. There's also an async streaming version (csv-parse) for huge files (millions of rows). For test fixtures, sync is simpler.

Sample CSV File

```
data/users.csv:

testId,testCase,username,password,expected,expectedError
L01,valid standard user,standard_user,secret_sauce,pass,
L02,valid problem user,problem_user,secret_sauce,pass,
L03,locked out user,locked_out_user,secret_sauce,fail,locked out
L04,wrong password,standard_user,wrong_pass,fail,do not match
L05,empty username,,secret_sauce,fail,Username is required
L06,empty password,standard_user,,fail>Password is required
L07,sql injection,"' OR '1'='1",x,fail,do not match
L08,name with comma,"Smith, John",x,fail,do not match
L09,unicode, ,secret_sauce,fail,do not match
L10,xss attempt,<script>x,secret_sauce,fail,do not match
```

Notice line 7 (L07) and line 8 (L08) — the values are quoted because they contain commas. csv-parse handles that correctly. The naive split-by-comma version turns row L08 into 7 fields instead of 6.

Wrap It in a Helper

```
// utils/csv.js
const fs = require("fs");
```

```
const { parse } = require("csv-parse/sync");

function readCsv(filepath) {
  const text = fs.readFileSync(filepath, "utf8");
  return parse(text, {
    columns: true,
    skip_empty_lines: true,
    trim: true,
    bom: true, // strip BOM if present (common with Excel-exported CSVs)
  });
}

module.exports = { readCsv };
```

The `bom: true` option strips the byte-order-mark that Excel sneakily adds when you "Save As CSV". Without it, the **first column header** silently becomes `testId` instead of `testId` — and `row.testId` will be undefined for every row. Maddening to debug.

4. The Mocha Pattern for Data-Driven Tests

The Pattern in Full

```
// tests/login.dd.spec.js
const { Builder } = require("selenium-webdriver");
const { expect } = require("chai");
const path = require("path");
const LoginPage = require("../pages/LoginPage");
const InventoryPage = require("../pages/InventoryPage");
const { readExcel } = require("../utils/excel");

const dataset = readExcel(path.join(__dirname, "..", "data", "login-data.xlsx"));

describe("Login (data-driven from Excel)", function () {
  this.timeout(20000);
  let driver, loginPage, inventoryPage;

  beforeEach(async () => {
    driver = await new Builder().forBrowser("chrome").build();
    loginPage = new LoginPage(driver);
    inventoryPage = new InventoryPage(driver);
    await loginPage.open();
  });

  afterEach(async () => {
    await driver.quit();
  });

  dataset.forEach((row) => {
    it(`[${row.testId}] ${row.testCase}`, async () => {
      await loginPage.login(row.username, row.password);

      if (row.expected === "pass") {
        expect(await driver.getCurrentUrl()).to.include("/inventory");
      } else {
        const err = await loginPage.getErrorMessage();
        expect(err).to.include(row.expectedError);
      }
    });
  });
});
```

Run `npm test`. You will see 12 separate test results in the console and in the `mochawesome` report — one per row.

Why This Works (and How Mocha Sees It)

When Mocha loads your spec file, it executes the top-level code synchronously. That includes:

1. The `describe(...)` call.
2. Inside the `describe`, `dataset.forEach(...)` runs **right then**. Each iteration calls `it(...)` once.
3. By the time Mocha is "done loading," it has registered 12 `it` blocks just like if you'd hand-written them.

The `async` keyword on the `it` callback is fine — Mocha awaits each callback when the test runs. The `forEach` itself is synchronous (it doesn't return a Promise; it just calls the callback once per element). That's exactly what we want — Mocha needs all its registered before it starts running tests.

The Test ID & testCase Convention

Every row should have:

- **testId** — a stable identifier like L01, REG_07. Useful for cross-referencing with bug tickets.
- **testCase** — a short human description.

Combine them in the test name: [L05] empty username. Now the mochawesome report's failure list reads like a QA bug catalog. Stakeholders can ask "did L05 pass last night?" and get a clean answer.

```
it(`[${row.testId}] ${row.testCase}`, async () => {
  /* ... */
});
```

Without testId, you get test names like should log in with valid credentials repeated 12 times in the report. Useless.

A Common Wrong Way: forEach Inside an it

This looks tempting and is **wrong**:

```
// ❌ DON'T DO THIS
it("should log in with all credentials", async () => {
  for (const row of dataset) {
    await loginPage.login(row.username, row.password);
    // ... assert ...
  }
});
```

Now Mocha sees **one** test that internally loops 12 times. Problems:

- The mochawesome report shows 1 pass/fail, not 12.
- If row 7 fails, rows 8-12 never run (the test bails out).
- Browser state leaks between iterations (you didn't get a fresh beforeEach per row).
- Failure messages don't tell you *which* row failed without manual logging.

The whole point of data-driven testing is **N tests, one block of code**. Keep the it inside the forEach, not the other way around.

4b. Where Data Lives — Project Structure

We're extending your Week 4 framework. Here's where new files go:

```
your-framework/
├── data/                               ← NEW THIS WEEK
│   ├── login-data.xlsx
│   ├── login-data.csv
│   └── registration-data.xlsx
├── pages/
│   ├── BasePage.js
│   ├── LoginPage.js
│   └── ...
├── tests/
│   ├── login.spec.js                 ← old hardcoded test (keep it)
│   ├── login.dd.spec.js             ← NEW — data-driven version
│   └── registration.dd.spec.js       ← NEW — homework
├── utils/
│   ├── driver.js
│   ├── excel.js                     ← NEW
│   └── csv.js                       ← NEW
├── .env
├── .gitignore
└── package.json
```

The .gitignore question: Test data with **fake** credentials is fine to commit (it's documentation of what you cover). Test data with real customer info, prod tokens, or PII goes in .gitignored files like data/secrets.xlsx. A common pattern:

```
# .gitignore
data/secrets.*
data/prod-*.csv
```

And keep a sanitized data/secrets.example.csv in version control so a teammate can copy it to data/secrets.csv and fill it in locally.

5. A Complete End-to-End Example

This is the one to read end-to-end, then come back to as a template.

data/login-data.xlsx (12 rows; same as the table in Section 2)

utils/excel.js

```
const XLSX = require("xlsx");

function readExcel(filepath, sheetName) {
  const wb = XLSX.readFile(filepath);
  const sheet = sheetName ? wb.Sheets[sheetName] : wb.Sheets[wb.SheetNames[0]];
  if (!sheet) throw new Error(`Sheet not found: ${sheetName} || "(first)"`);
  return XLSX.utils.sheet_to_json(sheet, { defval: "" });
}

module.exports = { readExcel };
```

pages/LoginPage.js (extending what you built in Week 1-4)

```
const { By } = require("selenium-webdriver");
const BasePage = require("../BasePage");

class LoginPage extends BasePage {
  static URL = "https://www.saucedemo.com";
  static USERNAME = By.id("user-name");
  static PASSWORD = By.id("password");
  static SUBMIT = By.id("login-button");
  static ERROR = By.css('[data-test="error"]');

  async open() {
    await super.open(LoginPage.URL);
  }

  async login(user, pass) {
    // Use sendKeys directly - empty string is a no-op, which is what we want
    // for empty-field test rows.
    await this.type(LoginPage.USERNAME, user || "");
    await this.type(LoginPage.PASSWORD, pass || "");
    await this.click(LoginPage.SUBMIT);
  }

  async getErrorMessage() {
    return this.getText(LoginPage.ERROR);
  }
}

module.exports = LoginPage;
```

tests/login.dd.spec.js

```
const { Builder } = require("selenium-webdriver");
const { expect } = require("chai");
const path = require("path");
const LoginPage = require("../pages/LoginPage");
const { readExcel } = require("../utils/excel");

const dataset = readExcel(path.join(__dirname, "..", "data", "login-data.xlsx"));

describe("Login - Data-driven (Excel)", function () {
  this.timeout(20000);
  let driver, loginPage;

  beforeEach(async () => {
    driver = await new Builder().forBrowser("chrome").build();
    loginPage = new LoginPage(driver);
    await loginPage.open();
  });

  afterEach(async () => {
    await driver.quit();
  });

  dataset.forEach((row) => {
    it(`[${row.testId}] ${row.testCase}`, async () => {
      await loginPage.login(row.username, row.password);

      if (row.expected === "pass") {
        const url = await driver.getCurrentUrl();
        expect(url).to.include("/inventory");
      } else {
        const err = await loginPage.getErrorMessage();
        expect(err).to.include(row.expectedError);
      }
    });
  });
});
```

Run It

npm test

Expected output:

```
Login - Data-driven (Excel)
✓ [L01] valid standard user (3210ms)
✓ [L02] valid problem user (3104ms)
✓ [L03] locked out user (2890ms)
✓ [L04] wrong password (2811ms)
✓ [L05] empty username (2670ms)
...
✓ [L12] trailing newline in pass (2790ms)

12 passing (38s)
```

Open `mochawesome-report/mochawesome.html` — all 12 tests show up as separate, named entries.

Demo: Building a 12-Row Data-Driven Login Test

Objective: Take ONE existing login test from your Week 4 framework. Replace the hardcoded credentials with rows from an Excel file. Watch the report grow from 1 test → 12 tests with no test-code duplication.

Pseudo-Code Template for Live Coding

```
// INSTRUCTOR: Have the framework from Week 4 open. The current login.spec.js
// has ONE hardcoded test. We're going to replace it.

// =====
// Step 1: Install xlsx (~2 min)
// =====
// INSTRUCTOR: Run live: npm install xlsx
// Show package.json updating. No imports yet.

// =====
// Step 2: Create data/login-data.xlsx (~5 min)
// =====
// INSTRUCTOR: Open Excel/Numbers/Sheets. Create headers in row 1:
// testId | testCase | username | password | expected | expectedError
//
// Add ~12 rows. Mix of pass/fail. Save as .xlsx in /data folder.
// Show the file in finder/explorer.

// =====
// Step 3: Build utils/excel.js (~3 min)
// =====
// INSTRUCTOR: Create utils/excel.js. Type out:
// const XLSX = require("xlsx");
// function readExcel(file) { ... }
// FILL: 4 lines - readFile, get sheet, sheet_to_json with defval, return.
// module.exports = { readExcel };

// =====
// Step 4: Quick sanity REPL (~2 min)
// =====
// INSTRUCTOR: Run a one-liner from terminal:
// node -e "console.log(require('./utils/excel').readExcel('data/login-data.xlsx'))"
// Should print 12 row objects. Confirm headers come through correctly.

// =====
// Step 5: Convert login.spec.js to data-driven (~5 min)
// =====
// INSTRUCTOR: Open tests/login.spec.js. Show the existing single it() block.
// Add at top:
// const { readExcel } = require("../utils/excel");
// const dataset = readExcel(...);
// FILL: Wrap the existing it() in dataset.forEach(row => ...).
// Replace hardcoded "standard_user"/"secret_sauce" with row.username/row.password.
// Update the expectation to branch on row.expected === "pass".
// Update the test name to include row.testId and row.testCase.

// =====
// Step 6: Run and admire (~3 min)
// =====
// INSTRUCTOR: npm test
// Watch 12 results. Open mochawesome report. Show each row as a named test.
// Open the failure list - point out how the testId makes triage trivial.

// =====
// Step 7: Add one more row LIVE (~2 min - the "wow" moment)
// =====
// INSTRUCTOR: Open the .xlsx, add row L13: a new edge case (e.g., 1000-char
```



```
// username). Save the file. Re-run npm test. 13 tests now run. NO CODE CHANGE.  
// THIS is the value of data-driven testing.
```

Expected End State

After the demo, students should see:

- A 12-row Excel file in /data
 - A `utils/excel.js` helper they can copy
 - A `tests/login.dd.spec.js` with one `forEach` and one `it` block — total length ~25 lines
 - A mochawesome report listing 12 (or 13!) separate test results
-

In-Class Practice (2 hours)

These are progressively harder. Pair-programming encouraged. Use your own Week 4 framework as the starting point.

Q1 (Easy) — Install and Verify

Install `xlsx` and `csv-parse` in your Week 4 framework. Run `npm ls xlsx csv-parse` and paste the output into a comment in your project's README. Both packages should appear without warnings.

Q2 (Easy) — A Tiny Spreadsheet

Create `data/practice.xlsx` with these 5 rows:

id name age

```
1 Alice 30  
2 Bob 25  
3 Carol 45  
4 Dan 19  
5 Eve 99
```

Write a script `scripts/dump-practice.js` that reads the file with `xlsx`, prints the rows, and prints the total count. No Selenium, no Mocha — just verify your reader works.

Q3 (Easy) — A Tiny CSV

Create `data/practice.csv` with the **same** 5 rows. Modify `scripts/dump-practice.js` to also print the rows from the CSV using `csv-parse`. Confirm both outputs match exactly.

Q4 (Easy) — Build the `readExcel` Helper

Create `utils/excel.js` with a `readExcel(filepath, sheetName?)` function as shown in Section 2. Add an optional second parameter for sheet name. Throw a clear error if the sheet doesn't exist. Export it.

Q5 (Medium) — Build the `readCsv` Helper

Create `utils/csv.js` with a `readCsv(filepath)` function using `csv-parse`. Include `bom: true` and `trim: true`. Export it. Use it in `scripts/dump-practice.js` instead of inlining `parse(...)`.

Q6 (Medium) — Data-Driven Login from CSV

Take ONE existing test from your framework (login is easiest). Create `data/login-data.csv` with at least **10 rows** including:

- 2 valid combinations
- 3 invalid combinations
- 1 empty-username case
- 1 empty-password case
- 1 SQL-injection-style string
- 1 unicode/special-character case
- 1 locked-out user (if your target supports it)

Write `tests/login.dd.spec.js` that uses `readCsv` and the `forEach/it` pattern. Add `testId` and `testCase` to each row. Run it. All 10 should pass (because the assertions match the expected column).

Q7 (Medium) — Verify the Report

Run `npm test` and open the mochawesome report. Confirm:

- All 10 rows appear as separate test entries
- Each test name starts with `[testId]` (e.g., `[L01]`)
- Test descriptions are meaningful at a glance
- Failures (if you intentionally break one row) point to the right `testId`

To intentionally break one: change expected for `L02` from pass to fail. Re-run. Confirm the report flags row `L02` as failing with a useful message.

Q8 (Medium) — Switch to Excel

Convert your Q6 CSV to an Excel file (`data/login-data.xlsx`). Update `tests/login.dd.spec.js` to use `readExcel` instead of `readCsv`. Run again. Same 10 tests should pass.

You now have **two** data sources for the same logical test. In `tests/login.dd.spec.js`, leave a comment: `// Source: data/login-data.xlsx (mirror in login-data.csv)`.

Q9 (Harder) — Two-Sheet Spreadsheet

Open `data/login-data.xlsx` and add a second sheet named `EdgeCases` with 5 weird rows (very long input, leading/trailing spaces, embedded newline, emoji, control characters). Modify your test to run BOTH sheets:

```
const main = readExcel("data/login-data.xlsx");
const edge = readExcel("data/login-data.xlsx", "EdgeCases");

describe("Login - Main", function () { main.forEach(row => it(...)); });
describe("Login - Edge Cases", function () { edge.forEach(row => it(...)); });
```

Run. Verify both describe blocks appear in the mochawesome report.

Q10 (Harder / Discussion)

In a 2-3 sentence comment in your test file, answer:

"What kinds of test data should NOT live in `/data` (and where should they live instead)?"

Examples to think about: real customer credentials, API tokens, environment-specific URLs, generated random emails, ephemeral test users created during the run.

Compare with your pair. Your instructor will discuss correct answers during wrap-up.

Homework Practice (Complete Before Next Class)

HW1 (Medium) — Registration Form Data-Driven Test

Pick a target site that has a registration form (suggestions: <https://practice.expandtesting.com/register>, <https://demoqa.com/automation-practice-form>, or <https://automationexercise.com>).

Build a `RegistrationPage` (extending `BasePage`, locators as static fields, methods describe intent — same conventions as Week 1). Then create `data/registration-data.xlsx` with **20 rows**.

Required columns: `testId`, `testCase`, `firstName`, `lastName`, `email`, `password`, `confirmPassword`, `expected`, `expectedError` (or similar).

Required mix:

- At least 5 valid registrations (vary first/last name shape, password content)
- At least 5 invalid emails (missing `@`, multiple `@`, no domain, no TLD, etc.)
- At least 3 password-mismatch cases (passwords don't match confirm)
- At least 2 password length boundaries (min - 1, min, max, max + 1 — pick which match the form)
- At least 2 missing-required-field cases
- At least 2 special-char-in-name cases (apostrophes, hyphens, accented chars, emoji)

Write `tests/registration.dd.spec.js` using the same pattern as the login test. Each row's expected column drives whether you assert success or a specific error.

Push to GitHub. Submit the repo link.

HW2 (Medium) — Source Switch

Make HW1 work from EITHER an Excel or a CSV file by reading an env var:

```
const source = process.env.DATA_SOURCE || "xlsx";
const dataset = source === "csv" ? readCsv("data/registration-data.csv") : readExcel("data/registration-data.xlsx");
```

Provide both data/registration-data.xlsx and data/registration-data.csv (same content). Verify DATA_SOURCE=csv npm test and DATA_SOURCE=xlsx npm test both produce 20 results.

HW3 (Medium) — Filtering

Add support for running only a subset of rows by testId prefix. E.g.:

```
TEST_FILTER=REG_VAL npm test    # only rows whose testId starts with REG_VAL
```

In your spec file:

```
const filter = process.env.TEST_FILTER || "";
const filtered = dataset.filter(row => row.testId.startsWith(filter));
filtered.forEach(row => it(...));
```

Useful for fast iteration: "I only care about validation rows right now."

HW4 (Harder) — A Helper That Handles Both

Refactor: replace readExcel and readCsv with one readData(filepath) in utils/data.js that picks the right parser based on the file extension.

```
function readData(filepath) {
  if (filepath.endsWith(".xlsx") || filepath.endsWith(".xls")) {
    return readExcel(filepath);
  }
  if (filepath.endsWith(".csv")) {
    return readCsv(filepath);
  }
  throw new Error(`Unsupported data file: ${filepath}`);
}
```

Update both your login and registration specs to use readData. The tests shouldn't care which file format you used.

HW5 (Stretch) — Data Validation

Before running the tests, validate the dataset itself:

- Every row has a non-empty testId
- Every testId is unique (no duplicates)
- Every row has a non-empty expected value of either pass or fail
- Every fail row has a non-empty expectedError

If any row fails validation, throw a clear error and skip running tests. This catches bad spreadsheet edits before you waste 3 minutes running 20 broken tests.

```
function validate(rows) {
  const ids = new Set();
  rows.forEach((row, i) => {
    if (!row.testId) throw new Error(`Row ${i + 2}: missing testId`);
    if (ids.has(row.testId)) throw new Error(`Duplicate testId: ${row.testId}`);
    ids.add(row.testId);
    if (!["pass", "fail"].includes(row.expected)) {
      throw new Error(`Row ${row.testId}: expected must be pass|fail, got ${row.expected}`);
    }
    if (row.expected === "fail" && !row.expectedError) {
      throw new Error(`Row ${row.testId}: fail rows need expectedError`);
    }
  });
}
```

(Row index $i + 2$ because spreadsheets are 1-indexed and row 1 is the header.)

Key Takeaways

✅ Data-driven testing = one flow, N inputs. The flow is code; the inputs are data. ✅ xlsx reads Excel files in 4 lines: readFile, get sheet, sheet_to_json, done. ✅ csv-parse is the right CSV library; the manual split(",") approach silently breaks on quoted fields and BOMs. ✅ The Mocha pattern: dataset.forEach(row => it(name, async () => {...})). The forEach is synchronous registration; the it callback is async. ✅ Don't put forEach *inside* an it block — that collapses N tests into 1. ✅ Every row should have testId and testCase columns. Test names like [L05] empty username are gold for triage. ✅ Real test data goes in /data. Sensitive data goes in

.gitignored files with a .example template committed.  Watch out for: leading-zero loss, BOM on CSVs, empty cell undefined, Excel auto-typing dates and numbers.

Next Week: Logging, Waits, Headless & CLI Execution. We'll add Winston for structured logs, replace any leftover `setTimeout` calls with explicit waits, run the suite headless, and add npm scripts (`test:smoke`, `test:headless`, `test:ci`) so the framework runs cleanly anywhere — including CI.